

Universidad Nacional Autónoma de México

Facultad de Ciencias

Seminario de Ciencias de la Computación B

Computación en la Nube

Tarea 5

Vista de Modelos, CRUD y Pruebas JUnit

Gustavo Márquez Flores

13 de mayo de 2026

Objetivo

Programar la vista de lista de modelos de autos, los métodos CRUD (Create, Read, Update y Delete) para la entidad de modelos de autos, y las pruebas JUnit de cada uno de estos métodos.

1. Vista de Lista de Modelos de Autos

Se implementó una vista web utilizando Thymeleaf y Bootstrap 5 que muestra la lista de modelos de autos registrados en la base de datos H2. La vista es accesible en la URL: <http://localhost:8001/modelos>

Se creó el controlador `ModeloViewController.java` con la anotación `@Controller` que carga todos los productos del servicio y los pasa al template `modelos.html` para su renderizado en tabla.

A continuación se muestra la vista de lista de modelos funcionando en el navegador:

The screenshot displays a web browser window at localhost:8001/modelos showing a table titled "Lista de Modelos de Autos". The table lists 10 car models with columns for ID, Marca, Modelo, Año, and Precio. The data is as follows:

ID	Marca	Modelo	Año	Precio
1	Ford	Focus	2010	\$250,000.00
2	Ford	Fusion	2016	\$350,000.00
3	GMC	Acadia	2026	\$1,352,000.00
4	GMC	Acadia	2026	\$1,352,900.00
5	GMC	Terrain	2026	\$802,900.00
6	Ford	Escape Híbrida	2025	\$801,100.00
7	Ford	Territory Híbrida	2025	\$719,900.00
8	Toyota	Corolla Cross HEV	2024	\$625,900.00
9	Toyota	Highlander HEV	2024	\$950,900.00
10	Tesla	Model 3	2026	\$800,000.00

The background of the browser window shows the IDE (IntelliJ IDEA) with the following code:

```
models.html
<html xmlns:th="http://www.thymeleaf.org">
</html>
<body class="bg-light">
<div class="container mt-5">
<div class="text-center mb-4">Lista de Modelos de Autos</div>
<div class="card shadow">
<div class="card-body">
<table class="table table-striped table-hover table-bordered">
<thead class="table-success">
<tr>
<th>ID</th>
<th>Marca</th>
<th>Modelo</th>
<th>Año</th>
<th>Precio</th>
</tr>
</thead>
<tbody>
<tr>
<td>1</td>
<td>Ford</td>
<td>Focus</td>
<td>2010</td>
<td>$250,000.00</td>
</tr>
<tr>
<td>2</td>
<td>Ford</td>
<td>Fusion</td>
<td>2016</td>
<td>$350,000.00</td>
</tr>
<tr>
<td>3</td>
<td>GMC</td>
<td>Acadia</td>
<td>2026</td>
<td>$1,352,000.00</td>
</tr>
<tr>
<td>4</td>
<td>GMC</td>
<td>Acadia</td>
<td>2026</td>
<td>$1,352,900.00</td>
</tr>
<tr>
<td>5</td>
<td>GMC</td>
<td>Terrain</td>
<td>2026</td>
<td>$802,900.00</td>
</tr>
<tr>
<td>6</td>
<td>Ford</td>
<td>Escape Híbrida</td>
<td>2025</td>
<td>$801,100.00</td>
</tr>
<tr>
<td>7</td>
<td>Ford</td>
<td>Territory Híbrida</td>
<td>2025</td>
<td>$719,900.00</td>
</tr>
<tr>
<td>8</td>
<td>Toyota</td>
<td>Corolla Cross HEV</td>
<td>2024</td>
<td>$625,900.00</td>
</tr>
<tr>
<td>9</td>
<td>Toyota</td>
<td>Highlander HEV</td>
<td>2024</td>
<td>$950,900.00</td>
</tr>
<tr>
<td>10</td>
<td>Tesla</td>
<td>Model 3</td>
<td>2026</td>
<td>$800,000.00</td>
</tr>
</tbody>
</table>
</div>
</div>
</div>
```

```
ModeloViewController.java
@Controller
@RequestMapping("/modelos")
public class ModeloViewController {
    private final IProductoService productoService;

    public ModeloViewController(IProductoService productoService) {
        this.productoService = productoService;
    }

    @GetMapping
    public ModelAndView listarModelos() {
        List<Producto> productos = productoService.listarProductos();
        ModelAndView mv = new ModelAndView("models", "productos", productos);
        return mv;
    }
}
```

2. Métodos CRUD

Se implementaron los cuatro métodos CRUD para la entidad de modelos de autos en el microservicio springboot-servicio-productos. Los endpoints REST son los siguientes:

- GET /listar — Obtiene todos los modelos
- GET /ver/{id} — Obtiene un modelo por ID
- POST /crear — Crea un nuevo modelo
- PUT /editar/{id} — Actualiza un modelo existente
- DELETE /eliminar/{id} — Elimina un modelo por ID

2.1 Read — Listar todos los modelos (GET /listar)

El endpoint GET /listar retorna la lista completa de modelos en formato JSON. Se puede observar que la base de datos contiene los modelos iniciales cargados desde import.sql:

The screenshot displays a web browser window at localhost:8001/listar showing a JSON array of car models. The IDE in the background shows the ProductControllerTest.java file with a test for the listar endpoint, and the terminal window displays the application logs.

Browser Response (GET /listar):

```
[{"id": 1, "marca": "Ford", "modelo": "Focus", "anio": 2010, "precio": 250000, "createAt": "2026-05-13"}, {"id": 2, "marca": "Ford", "modelo": "Fusion", "anio": 2016, "precio": 350000, "createAt": "2026-05-13"}, {"id": 3, "marca": "GMC", "modelo": "Acadia", "anio": 2026, "precio": 1352000, "createAt": "2026-05-13"}, {"id": 4, "marca": "GMC", "modelo": "Acadia", "anio": 2026, "precio": 1352900, "createAt": "2026-05-13"}, {"id": 5, "marca": "GMC", "modelo": "Terrain", "anio": 2026, "precio": 1352900, "createAt": "2026-05-13"}]
```

IDE Code (ProductControllerTest.java):

```
public class ProductControllerTest {  
    // Revertir  
    actualizado.setMarca(marcaOriginal);  
    productService.save(actualizado);  
}
```

Terminal Logs:

```
2026-05-13 23:04:15.928 INFO 25992 --- [freshExecutor-8] com.netflix.discovery.DiscoveryClient : Disable delta property : false
```

2.2 Create — Crear un nuevo modelo (POST /crear)

Se creó un nuevo modelo de auto mediante una petición POST al endpoint /crear con los datos en formato JSON. En el ejemplo se agregó un Tesla Model 3 2026 con precio \$800,000. La respuesta muestra el objeto creado con su ID asignado (id: 10):

The screenshot displays a web application running on localhost:8001/models. The page title is "Lista de Modelos de Autos". It contains a table with 10 rows of car data. The table has columns: ID, Marca, Modelo, Año, and Precio. The data includes cars from Ford, GMC, Toyota, and Tesla. The last row shows a Tesla Model 3 from 2026 with a price of \$800,000.00.

ID	Marca	Modelo	Año	Precio
1	Ford	Focus	2010	\$250,000.00
2	Ford	Fusion	2016	\$350,000.00
3	GMC	Acadia	2026	\$1,352,000.00
4	GMC	Acadia	2026	\$1,352,900.00
5	GMC	Terrain	2026	\$802,900.00
6	Ford	Escape Híbrida	2025	\$801,100.00
7	Ford	Territory Híbrida	2025	\$719,900.00
8	Toyota	Corolla Cross HEV	2024	\$625,900.00
9	Toyota	Highlander HEV	2024	\$950,900.00
10	Tesla	Model 3	2026	\$800,000.00

On the right, a REST client shows a successful POST request to the endpoint http://localhost:8001/crear. The request body is a JSON object: {"marca": "Tesla", "modelo": "Model 3", "año": 2026, "precio": 800000}. The response is a JSON object: {"id": 10, "marca": "Tesla", "modelo": "Model 3", "año": 2026, "precio": 800000.0, "createAt": "2024-10-10T10:10:10.000Z"}.

2.3 Update — Actualizar un modelo (PUT /editar/{id})

Se actualizó el modelo con ID 1 (Ford Focus) mediante una petición PUT. Se modificó el modelo a 'Focus Updated', el año a 2025 y el precio a \$300,000. La tabla refleja el cambio inmediatamente:

The screenshot displays a web browser on the left and a code editor on the right. The browser shows a page titled 'Lista de Modelos de Autos' with a table of car models. The table has columns for ID, Marca, Modelo, Año, and Precio. The first row shows 'Ford Focus Updated' with an ID of 1, year 2025, and price \$300,000.00. The code editor on the right shows a PUT request in a REST client, updating the car model with ID 1 to 'Focus Updated' with a price of 300000.0.

ID	Marca	Modelo	Año	Precio
1	Ford	Focus Updated	2025	\$300,000.00
2	Ford	Fusion	2016	\$350,000.00
3	GMC	Acadia	2026	\$1,352,000.00
4	GMC	Acadia	2026	\$1,352,900.00
5	GMC	Terrain	2026	\$802,900.00
6	Ford	Escape Híbrida	2025	\$801,100.00
7	Ford	Territory Híbrida	2025	\$719,900.00
8	Toyota	Corolla Cross HEV	2024	\$625,900.00
9	Toyota	Highlander HEV	2024	\$950,900.00
10	Tesla	Model 3	2026	\$800,000.00

```
PUT http://localhost:8001/editar/1
Content-Type: application/json
{"marca": "Ford", "modelo": "Focus Updated", "año": 2025, "precio": 300000.0}
```

2.4 Delete — Eliminar un modelo (DELETE /eliminar/{id})

Se eliminó el modelo con ID 1 mediante una petición DELETE al endpoint /eliminar/1. Al recargar la vista de modelos, el registro con ID 1 ya no aparece en la tabla:

The screenshot displays a web application running on localhost:8001. The browser window shows a list of car models with the following data:

id	marca	modelo	año	precio	createAt
2	Ford	Fusion	2016	350000	2026-05-13
3	GMC	Acadia	2026	1352000	2026-05-13
4	GMC	Acadia	2026	1352000	2026-05-13
5	GMC	Terrain	2026	802900	2026-05-13
6	Ford	Escape Híbrida	2025		

The terminal window shows the following REST client requests and responses:

```
(base) PS C:\Users\Edu\Documents\CC\Semestre 6\Nube\Computacion-En-La-Nube> Invoke-WebRequest -Method POST -Uri "http://localhost:8001/crear" -Content-Type "application/json" -Body '{"marca":"Tesla","modelo":"Model 3","año":2026,"precio":880000}'

id : 10
marca : Tesla
modelo : Model 3
año : 2026
precio : 880000.0
createAt :

(base) PS C:\Users\Edu\Documents\CC\Semestre 6\Nube\Computacion-En-La-Nube> Invoke-WebRequest -Method PUT -Uri "http://localhost:8001/editar/1" -Content-Type "application/json" -Body '{"marca":"Ford","modelo":"Focus Updated","año":2025,"precio":300000}'

id : 1
marca : Ford
modelo : Focus Updated
año : 2025
precio : 300000.0
createAt : 2026-05-13

(base) PS C:\Users\Edu\Documents\CC\Semestre 6\Nube\Computacion-En-La-Nube> Invoke-WebRequest -Method DELETE -Uri "http://localhost:8001/eliminar/1"
```

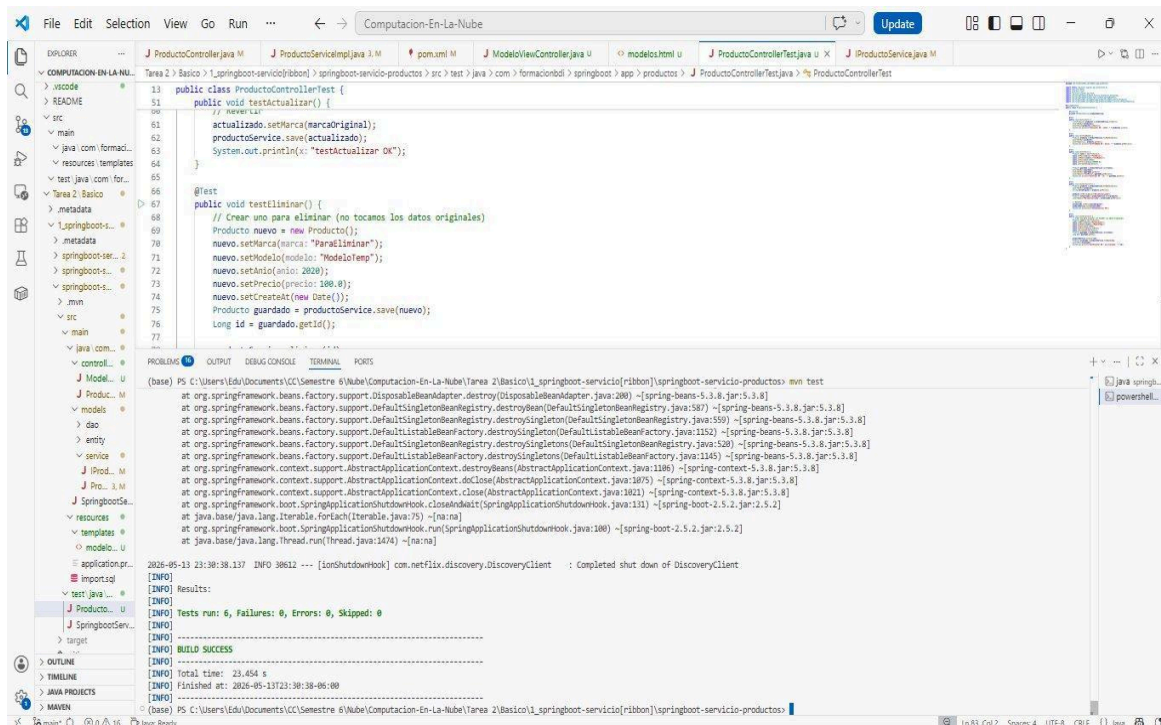

3. Pruebas JUnit

Se programaron 5 pruebas JUnit para cada uno de los métodos CRUD implementados en la clase `ProductoControllerTest.java`. Las pruebas utilizan `@SpringBootTest` para levantar el contexto completo de Spring e inyectar el servicio `IProductoService`.

Las pruebas implementadas son:

- `testListar()` — Verifica que la lista no sea nula y tenga elementos
- `testFindById()` — Verifica que se pueda obtener un modelo por ID
- `testCrear()` — Verifica que se pueda guardar un nuevo modelo y se asigne ID
- `testActualizar()` — Verifica que se pueda modificar un campo del modelo
- `testEliminar()` — Verifica que tras eliminar, el modelo ya no existe en BD

Al ejecutar `mvn test`, todas las pruebas pasan exitosamente con el resultado: Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 — BUILD SUCCESS:



```
public class ProductoControllerTest {
    public void testActualizar() {
        // never use
        actualizado.setMarca(marcaOriginal);
        productoService.save(actualizado);
        System.out.println("testActualizar OK");
    }

    @Test
    public void testEliminar() {
        // Crear uno para eliminar (no toques los datos originales)
        Producto nuevo = new Producto();
        nuevo.setMarca(marca: "ParaEliminar");
        nuevo.setModelo(modelo: "ModeloTemp");
        nuevo.setAnio(anio: 2020);
        nuevo.setPrecio(precio: 100.0);
        nuevo.setFecha(new Date());
        Producto guardado = productoService.save(nuevo);
        Long id = guardado.getId();
    }
}

(base) PS C:\Users\Edi\Documents\CC\Semestre 6\Nube\Computacion-En-La-Nube\Tarea 2\Basico\1_springboot-servicio[ritbon]\springboot-servicio-productos> mvn test
[INFO] Total time: 23.454 s
[INFO] Finished at: 2026-05-13T23:38:49-0800
[INFO] BUILD SUCCESS
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
```


Conclusión

Se implementaron exitosamente los tres requerimientos de la Tarea 5:

1. Vista de lista de modelos: Se creó una página web con Thymeleaf y Bootstrap 5 que muestra todos los modelos de autos en una tabla estilizada accesible en /modelos.
2. CRUD completo: Se implementaron los 5 endpoints REST (listar, ver, crear, editar, eliminar) en el controlador y servicio del microservicio springboot-servicio-productos.
3. Pruebas JUnit: Se programaron 5 pruebas unitarias que validan cada operación CRUD, todas pasando exitosamente con BUILD SUCCESS.